TP1 : Objet et mésures

Semestre 7 - durée : 2 h - Enseignant : T. Boris KIEMA

Objectifs. À la fin de cette séance, vous saurez :

- repérer les composants d'une carte à l'aide de sa datasheet ;
- lire un bouton sans perdre un seul appui ;
- convertir une tension en grandeur physique, calibrée et filtrée ;
- signaler une mesure invalide ;
- interroger un composant sur le bus I2C ;
- versionner votre code avec Git.

Exigences d'AgriSense couvertes : EX1 (mesurer au moins toutes les 15 min), EX2 (signaler une mesure invalide), EX8 (signaler un basculement du boîtier).

Matériel : Arduino Nano 33 IoT et câble micro-USB ; platine d'essai, fils, un bouton-poussoir, une LED, une résistance de 220 Ω ; une sonde capacitive d'humidité du sol (ou un potentiomètre) ; un verre d'eau.

Attention : La carte fonctionne en **3,3 V**. Une tension supérieure sur une broche la détruit. Vérifiez chaque câblage avant de brancher l'USB.

Parcours. Les exercices 0, 1, 2, 4 et 5 sont obligatoires et occupent les deux heures. L'exercice 3 et la question 3 de l'exercice 4 sont des bonus, à finir avant le TP2 si le temps manque.

Exercice 0 : prise en main et dépôt Git

20 min

1. Branchez la carte. Dans l'IDE Arduino, choisissez la carte « Arduino Nano 33 IoT » et son port, puis téléversez l'exemple *Blink*. La LED de la carte doit clignoter.
2. Ouvrez la datasheet de la carte (référence ABX00027), à mettre dans le dossier `docs/`. À l'aide de son schéma d'implantation, repérez sur votre carte : le microcontrôleur, le module radio, la centrale inertielle, la puce cryptographique, le connecteur USB et le bouton.
3. Dans la datasheet, relevez la tension de fonctionnement des broches. Que risque-t-on en y appliquant 5 V ?
4. Créez le dépôt. Le fichier `.ino` doit se trouver dans un dossier portant le même nom.

```
git init agrisense-gXX
cd agrisense-gXX
mkdir -p firmware/agrisense docs
printf "secrets.h\n" > .gitignore
git add . && git commit -m "TP1 : depot initial"
```

5. Enregistrez vos programmes sous `firmware/agrisense/agrisense.ino`. À quoi servira la ligne du fichier `.gitignore` au TP2 ?

Exercice 1 : GPIO et interruptions

30 min

Câblez le bouton entre la broche `D2` et `GND`.

Avant chaque téléversement. Écrivez à droite de chaque ligne ce qu'elle fait, en une phrase. Entourez toute ligne que vous ne savez pas expliquer et appelez l'enseignant.

1. Téléversez la version par scrutation. Le `delay(200)` simule un programme occupé à autre chose.

```
#include <Arduino.h>

const int BOUTON = 2;
uint32_t nbAppuis = 0;
bool etatPrecedent = HIGH;

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {} // attend le moniteur, 3 s au plus
  pinMode(BOUTON, INPUT_PULLUP);
  Serial.println("=== AgriSense TP1 ===");
}

void loop() {
  bool etat = digitalRead(BOUTON);
  if (etatPrecedent == HIGH && etat == LOW) { // passage de 1 a 0 : appui
    nbAppuis++;
    Serial.print("appuis : ");
    Serial.println(nbAppuis);
  }
  etatPrecedent = etat;
  delay(200);
}
```

2. Appuyez dix fois, très brièvement. Combien d'appuis sont comptés ? Pourquoi ?

3. Pourquoi un appui donne-t-il `LOW` et non `HIGH` ?

4. Que lirait-on avec `INPUT` au lieu de `INPUT_PULLUP` ?
   Essayez : remplacez `INPUT_PULLUP` par `INPUT` et remplacez 200 de delay par 20. Qu'observez-vous ?
   Approchez la main, faites bouger la carte. Qu'observez-vous ?
   Appuyez sur le bp sans le relâcher. Qu'observez-vous ?
   Après vos observations revenez au code initial pour la suite de cet exercice.

5. Remplacez la scrutation par une interruption, en conservant le `delay(200)`.

```
#include <Arduino.h>

const int BOUTON = 2;
volatile uint32_t nbAppuis = 0;

void surAppui() { nbAppuis++; }       // ISR : la plus courte possible

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}
  pinMode(BOUTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BOUTON), surAppui, FALLING);
}

void loop() {
  static uint32_t affiche = 0;
  if (nbAppuis != affiche) {
    affiche = nbAppuis;
    Serial.print("appuis : ");
    Serial.println(affiche);
  }
  delay(200);
}
```

6. Appuyez dix fois. Le compte est-il exact ? Est-il parfois supérieur à dix ? Nommez le phénomène.

7. Pour ignorer les rebonds, de quelles fonctions et déclarations avez besoin ? Corrigez dans l'ISR : ignorez toute transition qui survient moins de 50 ms après la précédente.

A titre d'information :
Il est possible que dans certains cas, vous ayez toujours 2 incrémentations au lieu d'une. Cela s'explique par le rebond au relachement du bouton. La technique consiste à détecter à regarder l'etat du bouton une fois qu'il s'est stabilisé.
L'ISR signale qu'il y a un front, et dans `loop()` on lit la broche quand rien n'a bougé depuis `20 ms`.
Pseudo-code :

```
//declaration
etatStable HIGH

appuiDetecte :            //routine ISR
    dernierFront <- capture du temps
    frontDetecte <- vrai
setup :
    ...
loop :
    si frontDetecte ET logique anti-rebond
       frontDetecte <- faux
       etat <- lire bouton
       si etat different de etatStable
        etatStable <- etat
        si etat = BAS
            nbAppui <- nbAppui + 1
            afficher
       fin si
    fin si
```

Exercice 2 : ADC, calibration et qualification

35 min

Câblez la sonde : `VCC` sur `3V3`, `GND` sur `GND`, `AOUT` sur `A0`. Avec un potentiomètre : les extrémités sur `3V3` et `GND`, le curseur sur `A0`.

1. Lisez la valeur brute chaque seconde, en 12 bits.

```
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}
  analogReadResolution(12);
}

void loop() {
  Serial.print("ADC brut : ");
  Serial.println(analogRead(A0));
  delay(1000);
}
```

2. Retirez `analogReadResolution(12)` : quelle plage obtient-on ? Calculez le pas de quantification en millivolts dans les deux cas.

3. Relevez la valeur brute à l'air libre, puis sonde plongée dans l'eau jusqu'au trait (**jamais l'électronique**). Reportez vos deux valeurs au tableau de la salle. Sont-elles identiques d'un binôme à l'autre ? Qu'en déduire pour un parc de 200 capteurs ?

4. Écrivez la conversion en pourcentage, bornée entre 0 et 100. La sonde étant capacitive, la valeur brute **diminue** quand l'humidité augmente.

```
const int SOL_SEC = 3100;   // VOTRE releve a l'air libre
const int SOL_HUMIDE = 1420; // VOTRE releve dans l'eau

float humiditeSol(int brut) {
  float pct = (float)(SOL_SEC - brut) * 100.0f / (float)(SOL_SEC - SOL_HUMIDE);
  return constrain(pct, 0.0f, 100.0f);
}
```

5. Touchez le fil de signal pour perturber la mesure et relevez trente valeurs. Programmez un filtre médian sur 9 échantillons, puis une moyenne sur 9, et comparez. Lequel résiste à un pic isolé ?

```
int mediane9(int* v) {          // trie le tableau, renvoie la valeur du milieu
  for (int i = 1; i < 9; i++) {
    int x = v[i], j = i - 1;
    while (j >= 0 && v[j] > x) { v[j + 1] = v[j]; j--; }
    v[j + 1] = x;
  }
  return v[4];
}
```

6. Débranchez le fil de signal, carte alimentée. Que lit-on ? Est-ce stable ? Programmez un indicateur de qualité `q` : 0 si la mesure est valide, 1 si elle est hors de la plage calibrée ou si les 9 échantillons sont trop dispersés. Justifiez vos seuils. (EX2)

Exercice 3 : PWM et temporisation non bloquante (bonus) 15 min

Câblez la LED et sa résistance de 220 Ω entre la broche `D5` et `GND`. Elle représente la vanne d'irrigation.

1. Faites varier l'intensité de la LED avec la sécheresse du sol : éteinte à 100 %, pleine puissance à 0 %.

```
analogWrite(5, map((int)sol, 0, 100, 255, 0));
```

2. Remplacez le `delay(1000)` de la boucle par une temporisation non bloquante. Vérifiez que les appuis sur le bouton sont toujours comptés pendant l'attente.

```
const uint32_t PERIODE = 1000;  // 1 s en TP ; 15 min sur le terrain
uint32_t derniereMesure = 0;

void loop() {
  if (millis() - derniereMesure >= PERIODE) {
    derniereMesure = millis();
    mesurer();
  }
}
```

3. `millis()` revient à zéro après 49 jours. Pourquoi l'écriture `millis() - derniereMesure >= PERIODE` reste-t-elle correcte, et pas `millis() >= derniereMesure + PERIODE` ?

Exercice 4 : datasheet et bus I2C 25 min

Matériel necessaire : aucun câblage car la centrale inertielle est soudée sur la carte et reliée au microcontrôleur par le bus I2C.

Téléchargez la datasheet de la centrale inertielle LSM6DS3, et placez la dans le dossier `docs/`. Relevez : (a) ses tensions d'alimentation minimale et maximale ; peut-on la brancher en 5 V ? (b) ses deux adresses I2C possibles ; (c) les plages de mesure de l'accéléromètre.

1. Téléversez le scanner. Combien de périphériques répondent, et à quelles adresses ? Laquelle correspond à votre réponse (b) ?

```
#include <Arduino.h>
#include <Wire.h>           //biblio. pilotant le bus I2C

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}
  Wire.begin();
  for (uint8_t adr = 1; adr < 125; adr++) { //Bus SPI est encode sur 7 bits
    Wire.beginTransmission(adr);
    if (Wire.endTransmission() == 0) {      //si l'esclave repond, il retourne 0
      Serial.print("peripherique a l'adresse 0x");
      Serial.println(adr, HEX);
    }
  }
}

void loop() {}
```

2. Lisez l'accélération sur les trois axes avec la bibliothèque `Arduino_LSM6DS3`. Carte à plat et immobile, que valent x, y et z ? Pourquoi z vaut-il environ 1 ?

```
#include <Arduino.h>
#include <Arduino_LSM6DS3.h>

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}
  if (!IMU.begin()) Serial.println("centrale inertielle absente");
}

void loop() {
  float x, y, z;
  if (IMU.accelerationAvailable()) {
    IMU.readAcceleration(x, y, z);      // en g
    Serial.print(x); Serial.print(" ");
    Serial.print(y); Serial.print(" ");
    Serial.println(z);
  }
  delay(200);
}
```

3. (bonus) Calculez l'inclinaison de la carte par rapport à la verticale, en degrés, et affichez une alerte au-delà de 30° (EX8).

$$\text{inclinaison} = \arccos\left(\frac{z}{\sqrt{x^2+y^2+z^2}}\right)$$

Exercice 5 : intégration et commit

10 min

1. Réunissez les exercices dans `agrisense.ino`. Le programme affiche une ligne par seconde, sous ce format :

```
sol=47.3 q=0 incl=2.1 appuis=3
```

2. Enregistrez la séance dans le dépôt.

```
git add .
git commit -m "TP1 : acquisition, qualification, inclinaison"
```

3. Complétez `README.md` : le câblage, vos deux valeurs de calibration, et la manière de téléverser.